

Analyse et conception objet de logiciels

*Abdelkrim Amani , N'NANE Dieudonne Junior ,
SAMAKE Habibata , SGHIER Anas , SMITH Syméon*

Analyse :

I. Diagramme des cas d'utilisations

Dans la figure 1, nous expliquons les différents cas d'utilisation énumérés sur le diagramme.

Une analyse du texte nous montre que nous aurons principalement :

A. acteurs secondaires :

On a un seul acteur, c'est le système de base de données. En effet cet acteur peut agir dans les décisions que notre système peut prendre (l'authentification, la vérification de l'éligibilité ..)

B. acteurs principaux :

On a deux acteurs principaux : les auteurs et les lecteurs. Parmi les auteurs, nous retrouvons ceux qui ont commencé la rédaction d'une histoire, c'est-à-dire l'auteur du premier paragraphe.

Les différents cas d'utilisations de l'application relève donc de 2 niveaux:

1. Les auteurs :

Afin de pouvoir rédiger un paragraphe, il faut au préalable être enregistré sur l'application. Suite de laquelle, tout auteur peut:

- ❖ créer une nouvelle histoire tout en définissant le mode de rédaction qui soit être sur invitation soit ouverte à tous les autres auteurs. La création d'une nouvelle histoire implique que l'auteur devra procéder à la rédaction d'un premier paragraphe qui se solde par l'indication des choix possibles(ceci étant pour tous les paragraphes), à la suite de laquelle il soumet une validation ou encore annuler sa rédaction.

- ❖ Lorsqu'une histoire se trouve sous le mode invitation(en écriture) l'auteur du premier paragraphe est en capacité d'inviter d'autres utilisateurs.

- ❖ L'auteur du premier paragraphe d'une histoire est le seul à pouvoir publier ou dépublier celle-ci. Cependant la publication requiert que l'histoire en question ait au moins une conclusion.

- ❖ Tout auteur a le droit de rajouter un choix à un paragraphe donné, de modifier un paragraphe si et seulement si il en est éligible, ou encore de supprimer un paragraphe si et seulement s'il en est éligible et que ce paragraphe-ci ne mène à aucun autre.

2. Les lecteurs:

Les lecteurs quant à eux ont un accès restreint lorsqu'ils ne sont pas enregistrés sur l'application. Ils peuvent néanmoins réaliser certaines actions à savoir:

- ❖ Choisir un paragraphe à lire
- ❖ parcourir une histoire(mode lecture)
- ❖ Enregistrer sa progression, possible que lorsque le lecteur est enregistré dans la base de donnée
- ❖ accéder à son historique

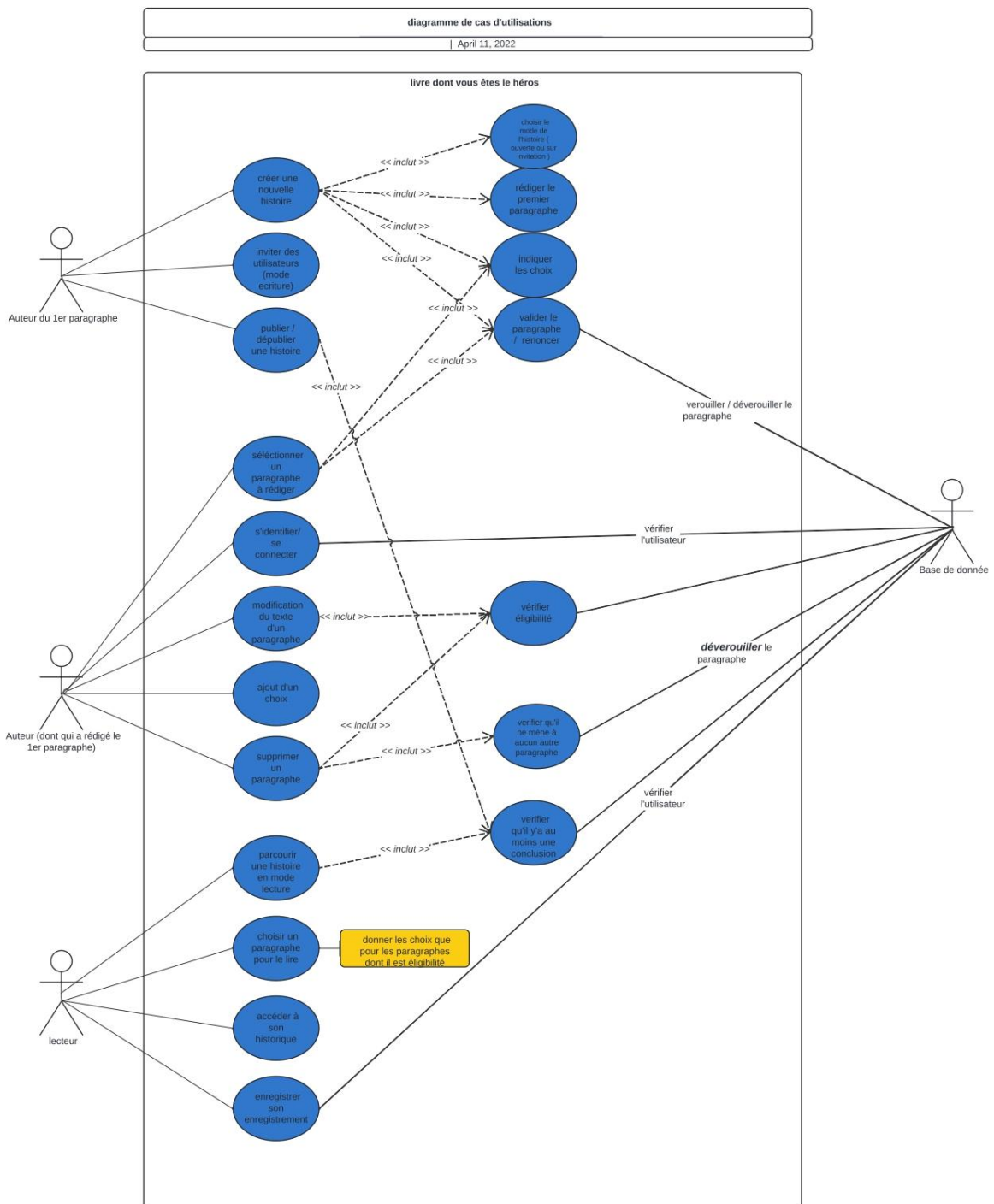


figure 1 : diagramme de classe

II. diagramme de séquence :

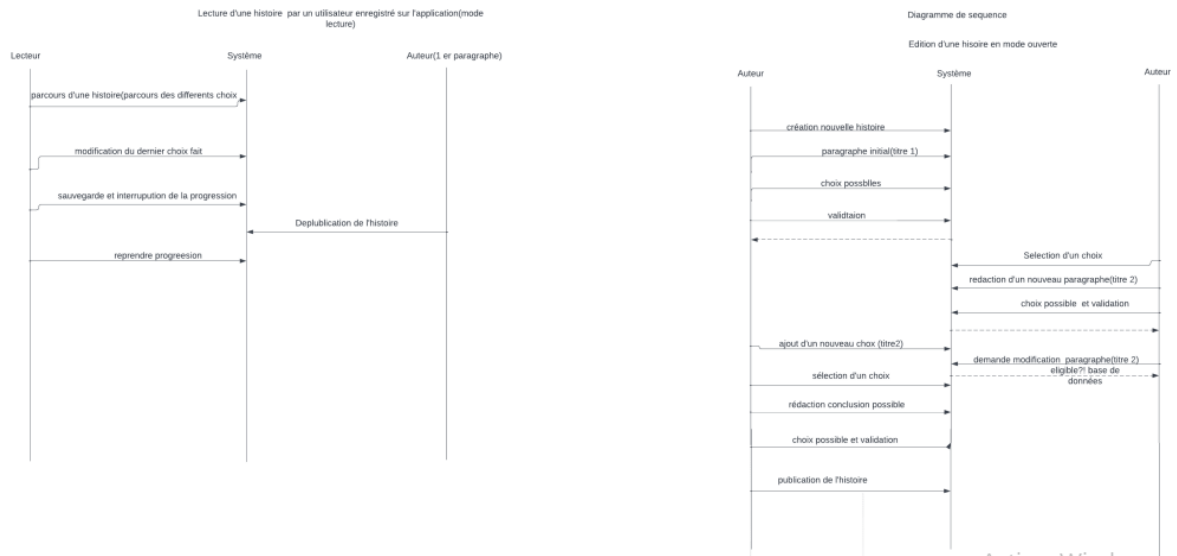


figure 2 : diagramme de séquence ; lecture d'une histoire par un utilisateur enregistré sur l'application (mode lecture), édition d'une histoire en mode ouverture

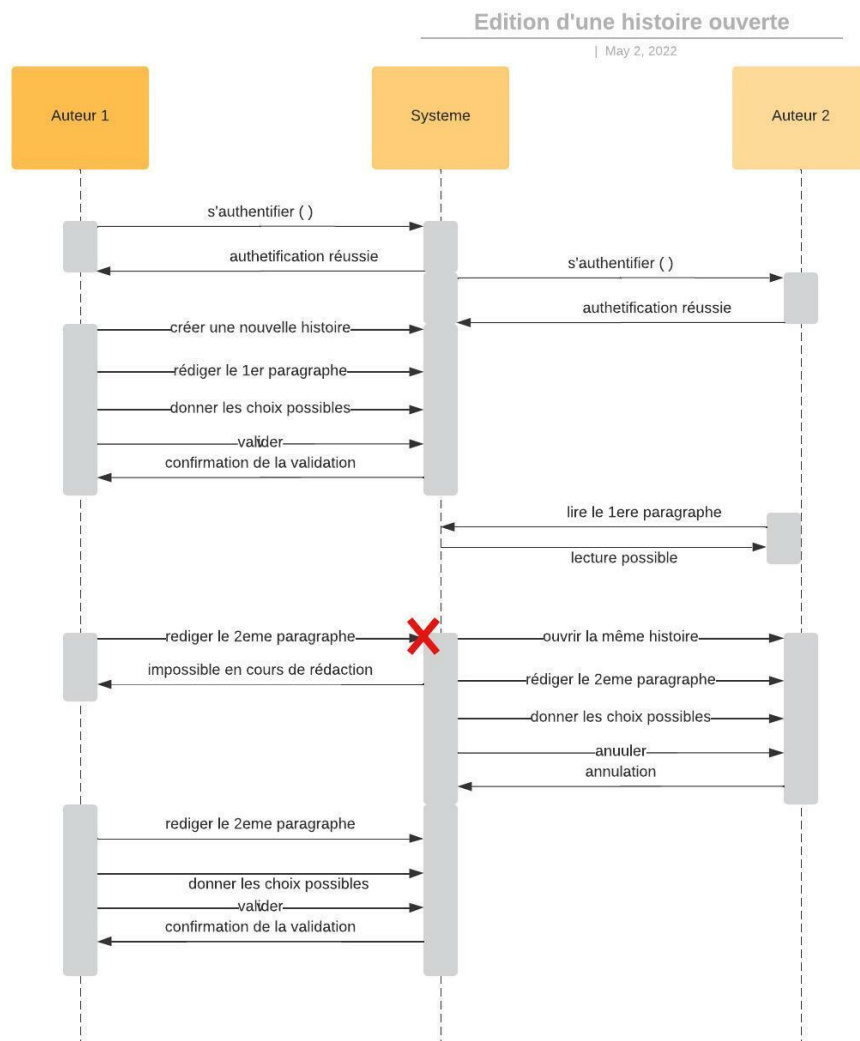


figure 2' : édition d'une histoire ouverte et annulation de rédaction

III. diagramme de classes d'analyse :

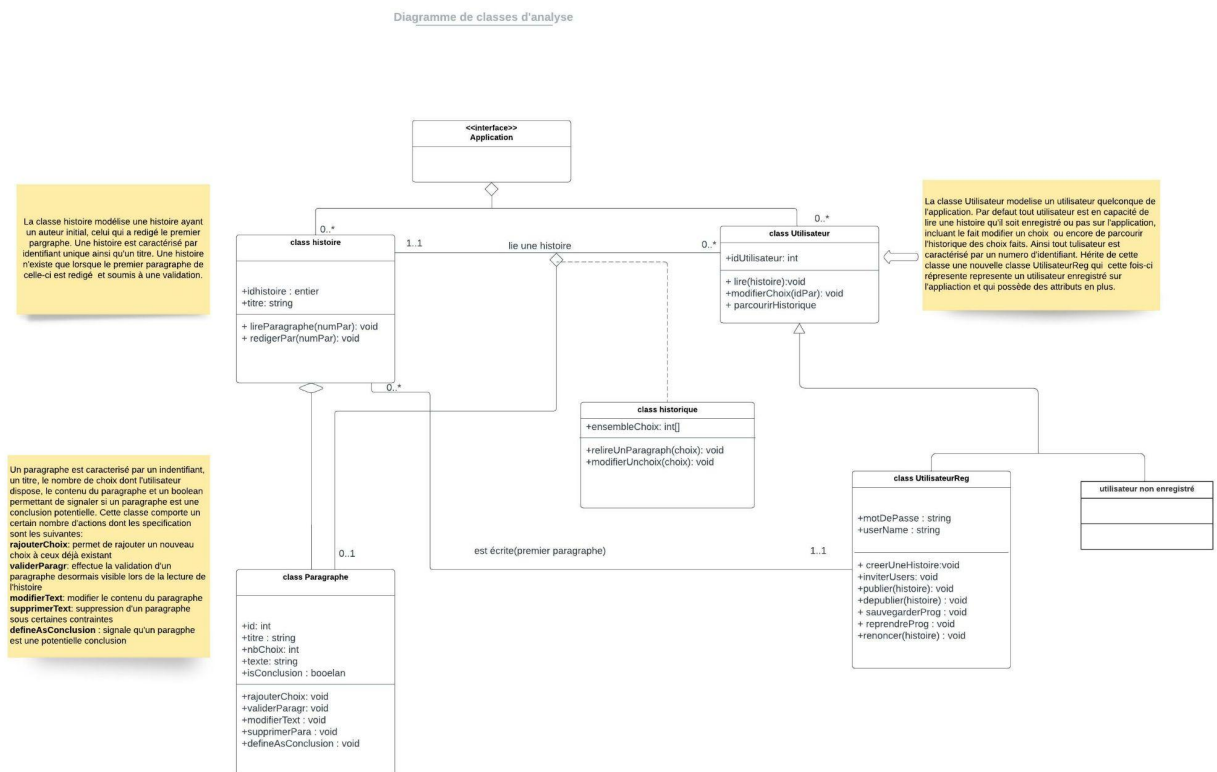


figure 3 : diagramme de classe d'analyse

Conception

IV. modèle-vue-contrôleur:

D'après le diagramme de classe d'analyse que nous disposons nous pouvons regrouper ces différentes classes en :

A. modèle :

Dans notre conception, dans la partie modèle, on se contente de mettre en place la base de données regroupant toutes les informations concernant les utilisateurs, les histoires, les paragraphes, etc, ainsi que les liens qui unissent ces différentes structures de données.

B. Vue :

Cette partie concerne le frontend du projet, ie ,tout ce qui concerne l'interface utilisateur de l'application.

C. contrôle :

Dans cette partie, le contrôleur est le serveur de notre application web. Nous indiquons comment devrait marcher chaque structure de données tout en explicitant les différentes fonctions de mise à jour de la base de données, le déroulement de l'application(des appels de fenêtre en fonction des différentes interactions.

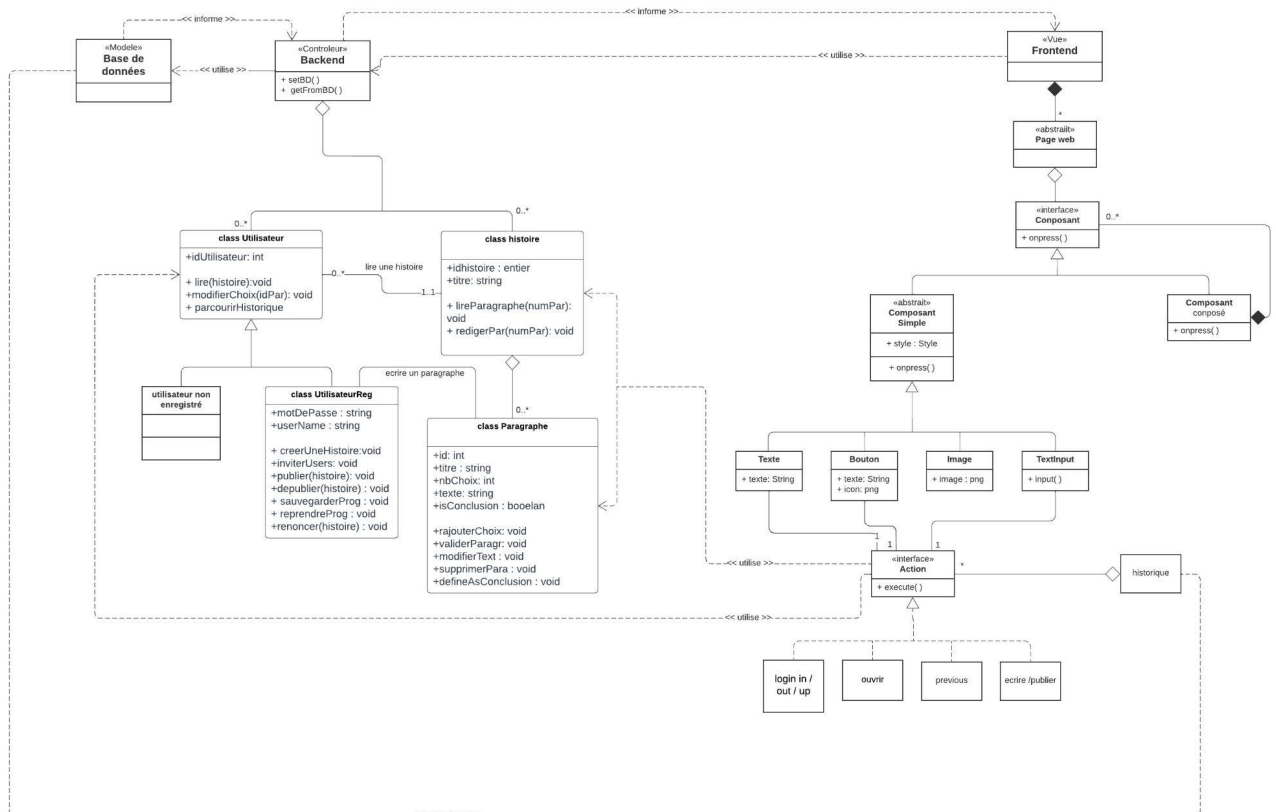
V. Diagramme de classe logicielles:

Dans ce diagramme (figure 4), On se base sur le modèle MVC (modèle , vue , contrôleur) . on a essentiellement 3 classes ;

La base de données (modèle) , Frontend (vue) , Backend (contrôleur) .

Il est dérivé du diagramme de classe d'analyse mais ici on y rajoute des classes émanant du frontend telles que les boutons, les logos, les textinput,... tout en indiquant les actions qu'ils font. On ajoute également une classe pour les pages web (Home Page , SignIn Page ...).

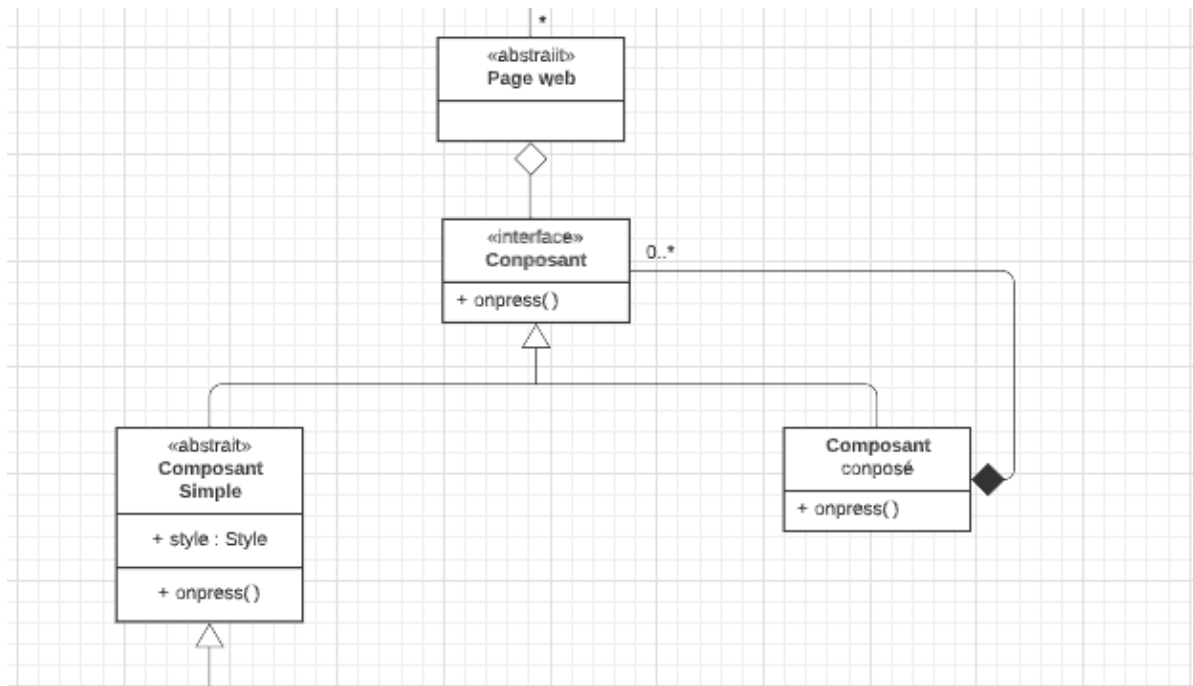
diagramme de classes logicielles



A. utilisation des patrons :

1. Patron Composite :

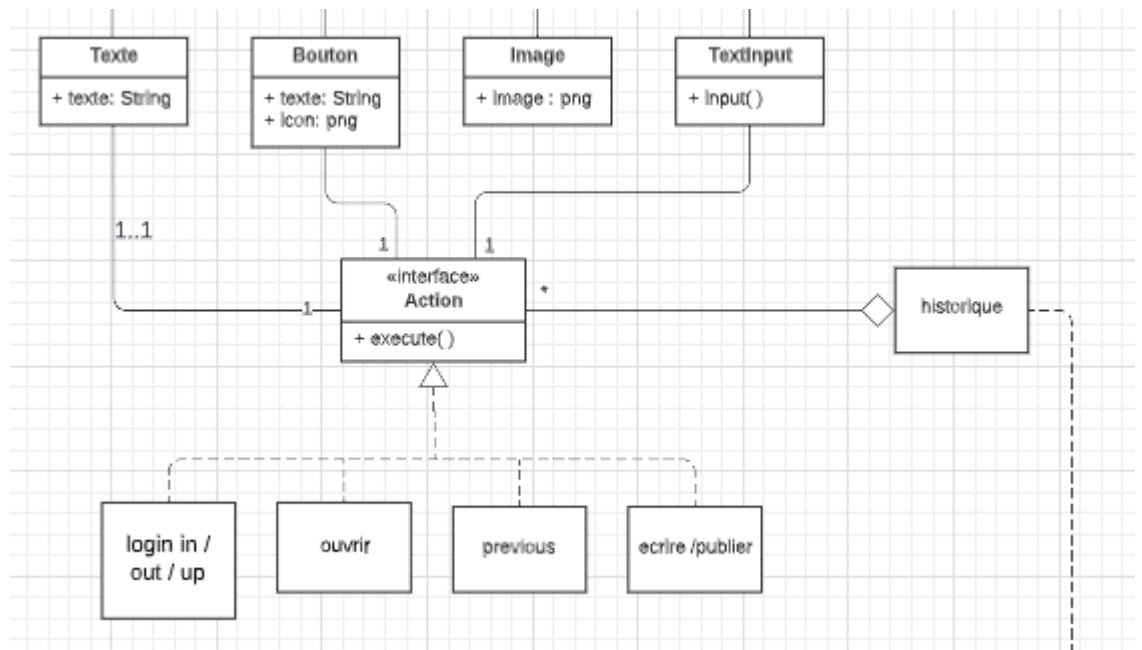
Pour une meilleure présentation des pages web on a choisi de les représenter à l'aide de ce patron . En effet , chaque page web présente un ensemble de composants qui peuvent être simples (texte , image ...) ou composés . Pour réaliser une tâche (dans notre cas c'est la méthode onpress()), un composant délègue cette tâche à ces sous-éléments.



2. Patron Commande :

Notre application web comporte plusieurs boutons. Chacun a une fonctionnalité bien déterminée. Par exemple retour à la page précédente, se connecter , ouvrir ...

Le retour en arrière nécessite l'enregistrement d'historique .



3. Patron Observateur:

Ce patron garantit à l'observateur d'envoyer des signaux de mise à jour et de recevoir d'autres qui informe de l'état du Sujet. généralement ce modèle est utilisé pour la conception de l'architecture MVC et tel est notre cas. En effet les classes du frontend communique avec la base de données à travers le backend. Ils ont accès aux informations stockées en BD(<< utilise >>) et suite à des modification faite grâce aux composants de nos pages (ajout d'utilisateur , ajout d'histoire , supprimer un paragraphe

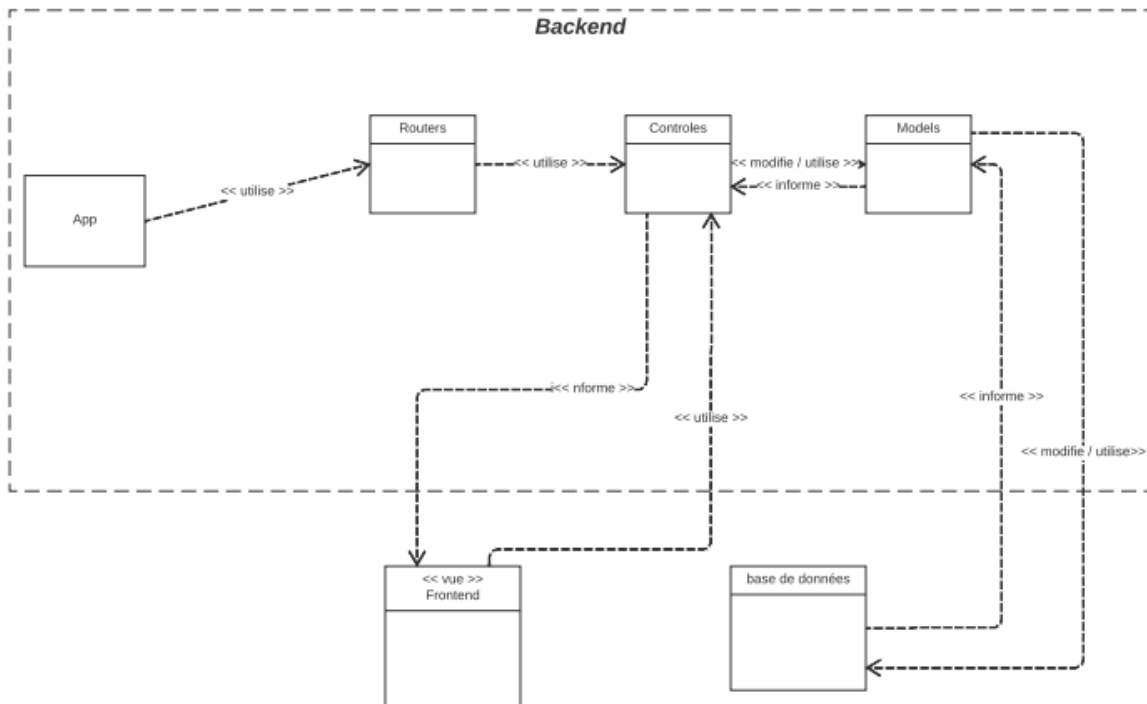
...), la classe frontend (et ses sous-classes) informe la classe backend pour qu' elle informe à son tour la base de donnée et fait les changement nécessaires.



B. Diagramme de classe Backend :

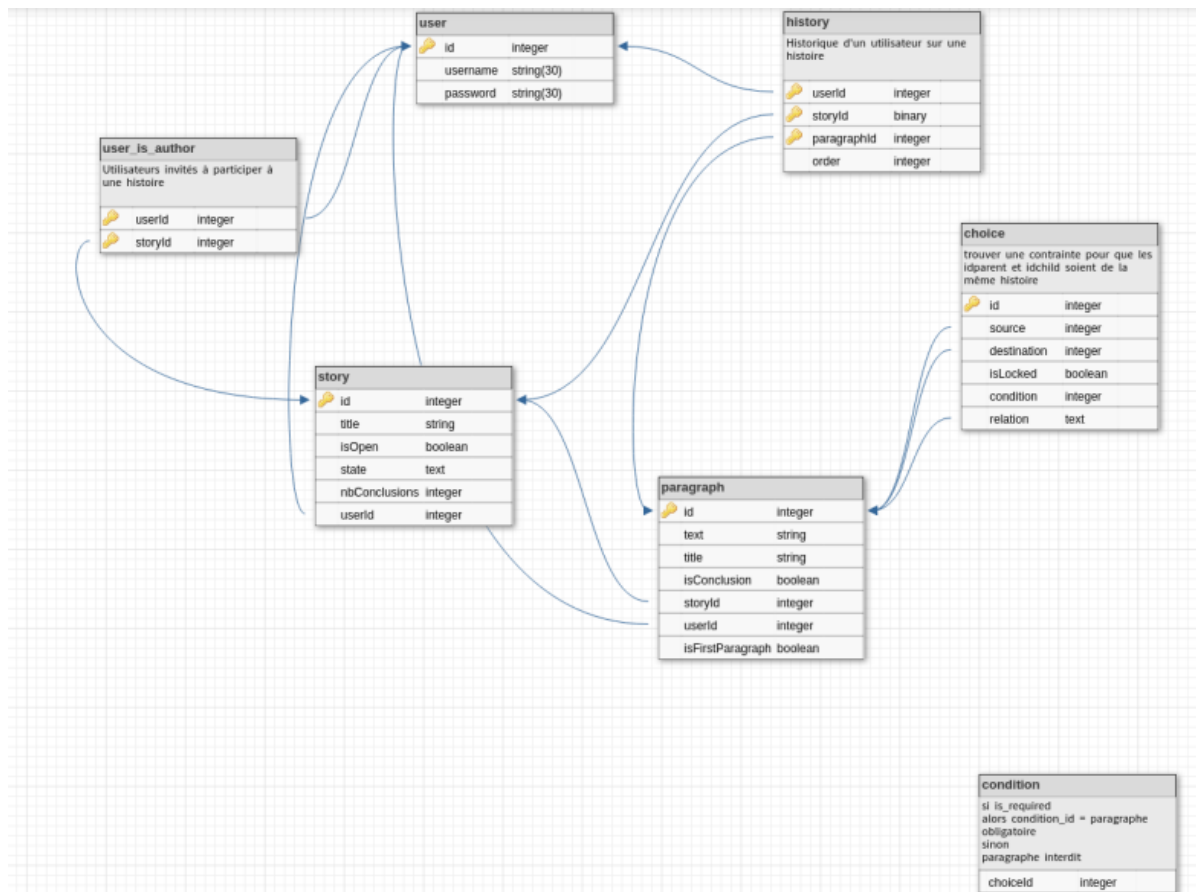
Pour une meilleure compréhension du sujet on a opté pour faire une conception un peu générale de la partie backend pour garantir qu'elle soit bien structurée et efficace.

Diagramme de classe du Backend

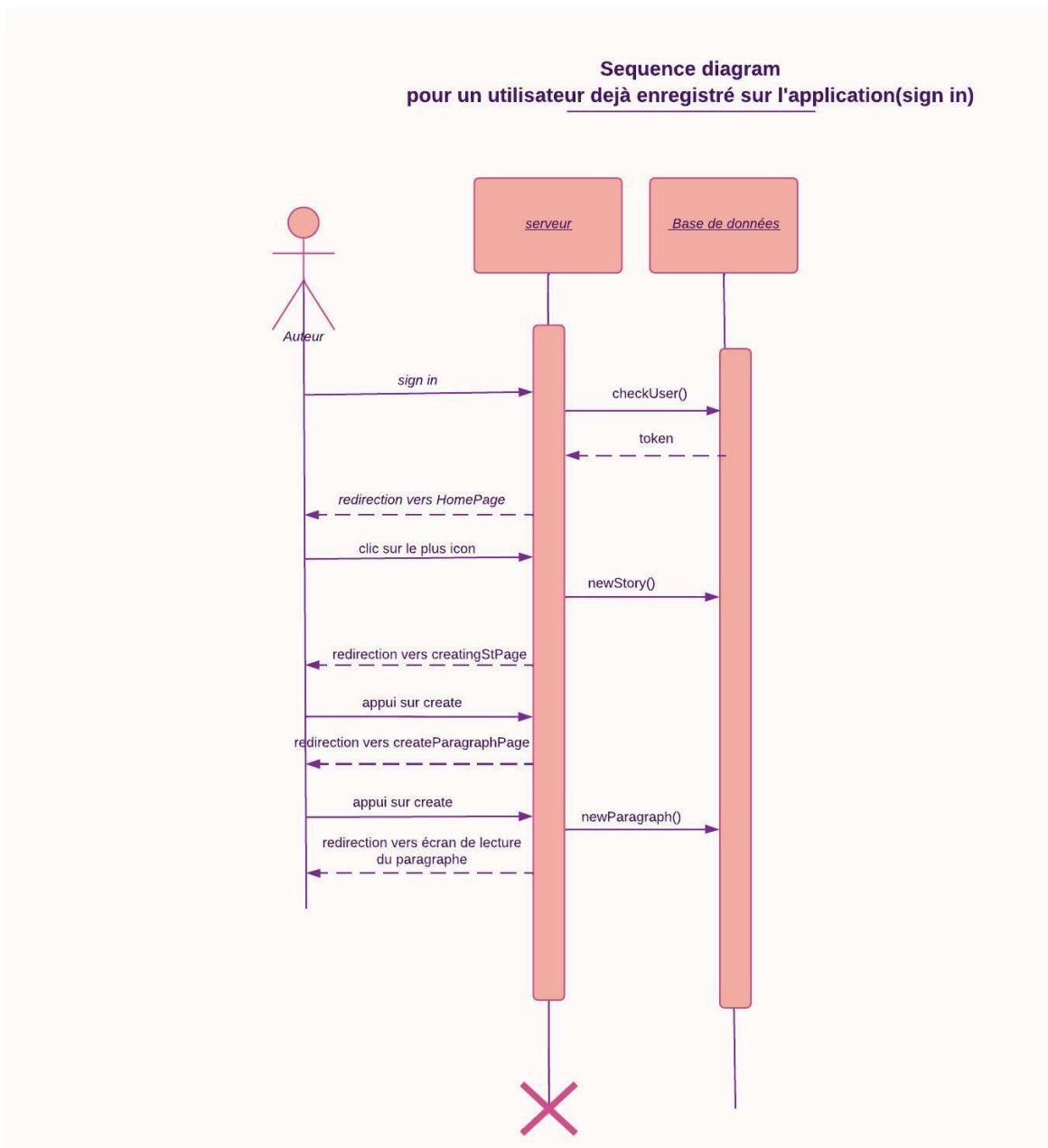


C. Diagramme de l'architecture de la base des données :

Ce diagramme est la clé pour la bonne compréhension de notre application.

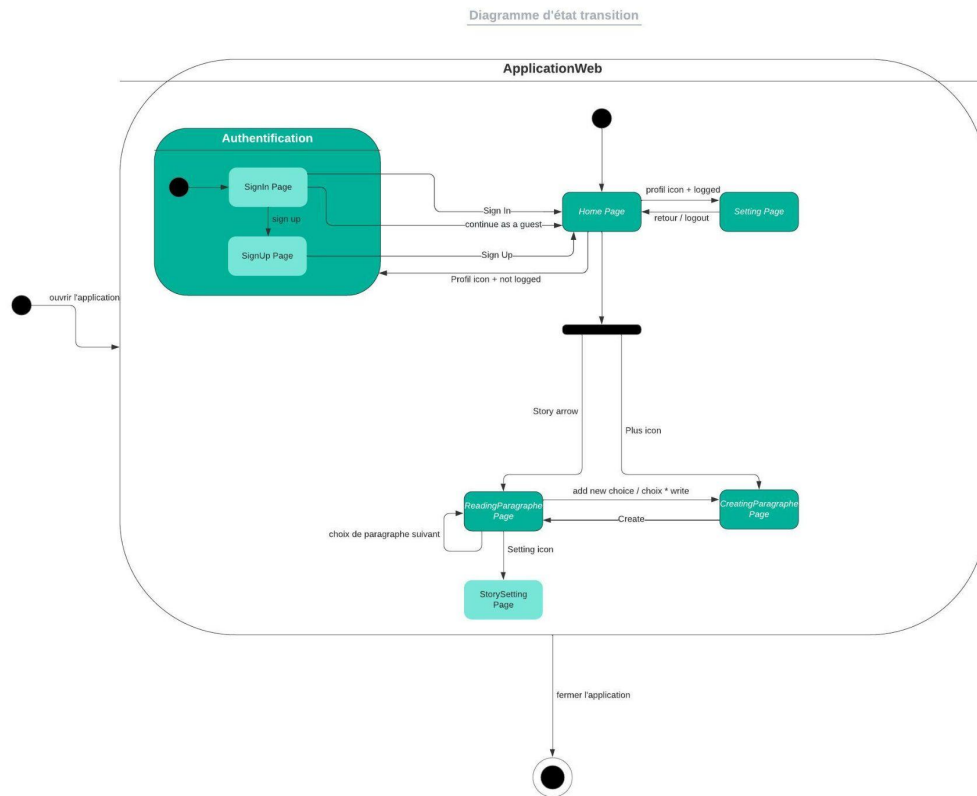


VI. Diagrammes de séquence:



Le diagramme de séquence (figure) permet d'illustrer les différentes actions des différents objets pour le cas d'utilisation "Écrire une nouvelle histoire par un utilisateur déjà enregistré sur l'application". Il fait intervenir les acteurs suivants: auteur, base de données et serveur. toute action qu'un utilisateur fait appel à une fonction qui sera exécutée par la base de données. Nous avons par exemple, l'action "cliquer sur l'icône plus" qui engendre l'appel de la fonction **newStory()**.

VII. Diagrammes d'état-transitions:



Afin de mettre en place un diagramme d'états transitions , nous avons principalement deux états **ReadingParagraphe** pour lire un paragraphe et **WritingParagraphe** pour rédiger un paragraphe. En effet, lorsqu'un utilisateur est en mode lecture il ne peut que se trouver dans l'état **reading**, état dans lequel il a la possibilité de rajouter un choix, ce qui peut l'amener dans l'état **writing**. Cependant pour pouvoir accéder à ces états, il faut passer par les états **Authentication** dans lequel l'utilisateur peut soit s'enregistrer(sign in) soit s'inscrire sur l'application, à la suite de laquelle, il passe dans l'état **HomePage** dans lequel il peut soit éditer une histoire soit faire une lecture, ou encore aller dans les paramètres **SettingPage** en se connectant.